

```

1 import math, numpy, pygame, obstacle, guy, res, random
2 from copy import copy
3 from scipy import *

5 game.window = pygame.window.Window(900, 750)
  white = (255,255,255,255)
7 pygame.gl.glClearColor(*white)
  main_batch = pygame.graphics.Batch()
9
10 #coordinate of the goal and obstacle
11
12 x_coord = [random.randint(1, 900) for i in range(8)]
13 y_coord = [random.randint(1, 750) for i in range(8)]
  coord = zip(x_coord, y_coord)
15
16 goal = [820,720]
17 target = pygame.sprite.Sprite(img=res.goal, x=goal[0], y=goal[1],
  batch=main_batch)

19 obs = [pygame.sprite.Sprite(img=res.obstacle, x=x_coord[i], y=
  y_coord[i], batch=main_batch) for i in range(1, len(x_coord)-1)]

21 objet = pygame.sprite.Sprite(img=res.player, x=x_coord[0], y=y_coord
  [0], batch=main_batch)

23
24 def dist(a,b):
25     return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)

27
28 @game.window.event #tells the window to draw objects
29 def on_draw():
  game.window.clear()
31   main_batch.draw()

33 extlen = 10
  def ef(src, target):
35     global extlen
  #extend as far as possible without colliding
37
38     A = 50
39
40
41     direction = [goal[i]-src[i] for i in xrange(len(src))]
  normdir = [direction[i]/dist([0,0], direction) for i in [0,1]]
43     return [src[0] + extlen*normdir[0], src[1] + extlen*normdir[1]],
  normdir

45 def mdist_obs(obs, newpt):
  mdist = min(dist(newpt, [obs1.x, obs1.y]) for obs1 in obs)
47   for obs1 in obs :
  if dist(newpt, [obs1.x, obs1.y]) == mdist :
49     nearobs = [obs1.x, obs1.y]
  return mdist, nearobs

51
52 def ef1(src, nearobs):
53     global extlen
  d = dist(src, nearobs)
55     normdir = [-(src[1]-nearobs[1])/d, (src[0]-nearobs[0])/d]
  return [src[0] + extlen*normdir[0], src[1] + extlen*normdir[1]],
  normdir
57

```

```

scr=coord[0]
59 d=dist(scr,goal)
normdir=[(scr[0]-goal[0])/d,(scr[1]-goal[1])/d]
61 def update(dt):
    global scr, objet, target, coord, obs, normdir
63 #while dist((objet.x, objet.y), goal)<=10:
    newpt,normdir=ef(scr, target)
65 mdist,nearobs = mdit_obs(obs,newpt)
    if mdist > 65:
67         objet.rotation = -270*math.atan2(normdir[0],normdir[1])/math.pi
        objet.x, objet.y=newpt[0],newpt[1]
69         scr=[objet.x, objet.y]
    else :
71         newpt,normdir=ef1([objet.x, objet.y], nearobs)
        objet.rotation = 100*math.atan2(normdir[0],normdir[1])/math.pi
73         objet.x, objet.y=newpt[0],newpt[1]
        scr=[objet.x, objet.y]
75     if dist(scr, goal)<=80:
        return
77
79
81 if __name__ == "__main__":
    # Update the game 120 times per second
    pyglet.clock.schedule_interval(update, 1/5.0)
83 #pyglet.clock.schedule_interval(dispatch, 0.2)
    # Tell pyglet to do its thing
85     pyglet.app.run()

```

robot1.py